



IPD434 · Seminario de Soft Computing

Computación evolutiva

De la formulación de un problema a la evolución de soluciones

Dr. Patricio Olivares Roncagliolo

Material basado y ampliado a partir del material desarrollado por el **Prof. Nicolás Gálvez Ramírez**.

Propósito de esta unidad

En IPD434 estudiamos métodos de *soft computing* cuando una solución exacta es costosa, desconocida o innecesaria.

En esta unidad la pregunta es:

¿Cómo buscar buenas soluciones cuando podemos evaluarlas, pero no sabemos construir directamente la mejor?

La computación evolutiva responde manteniendo y transformando una **población** de soluciones candidatas.

Conexión con la asignatura

Unidad 1 · Soft Computing

Reconocimos problemas con incertidumbre, grandes espacios de decisión y soluciones aproximadas.

Unidad 2 · Sistemas difusos

Incorporamos conocimiento experto para convertir entradas en decisiones interpretables.

Esta unidad · Computación evolutiva

Buscamos configuraciones de decisiones que optimicen una medida de calidad.

Unidad posterior · Redes neuronales

Aprenderemos parámetros desde datos, usualmente mediante optimización basada en gradiente.

Aquí no aprendemos una regla ni una función de predicción: diseñamos una búsqueda para hallar buenas configuraciones.

Resultados de aprendizaje

Al finalizar, se espera poder:

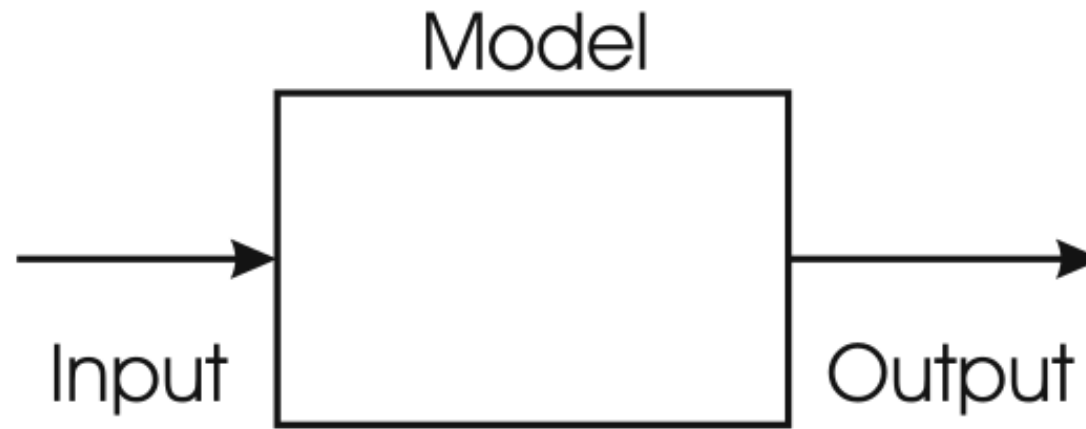
1. Formular un problema como optimización, satisfacción de restricciones o ambos.
2. Elegir una representación coherente con el dominio y las restricciones.
3. Explicar el equilibrio entre exploración, explotación y diversidad.
4. Diseñar selección, recombinación, mutación y supervivencia.
5. Implementar y analizar un algoritmo evolutivo para un problema combinatorio.
6. Evaluar un método estocástico con presupuestos y repeticiones comparables.

Ruta conceptual

Problema → espacio de búsqueda → metaheurística → población → evidencia

Cada etapa restringe la siguiente: una mala formulación o representación no se arregla agregando más generaciones al algoritmo.

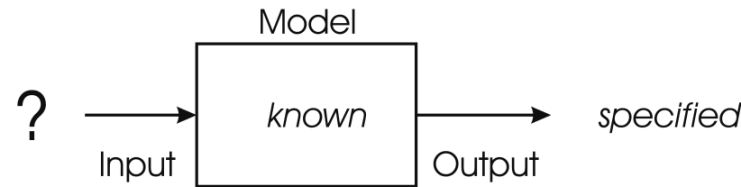
Resolver problemas como una caja negra



Un sistema puede verse como una relación entre **entrada**, **modelo** y **salida**. Según cuál de estos elementos conocemos y cuál deseamos encontrar, cambia el tipo de problema.

Problemas de optimización

Conocemos el modelo y la forma de evaluar su salida. La incógnita es la **entrada** que produce el mejor resultado.



En IPD434 podemos pensar en la planificación académica:

- **Modelo:** asignación de cursos a profesores, salas y bloques.
- **Entrada buscada:** una configuración particular de esas asignaciones.
- **Salida evaluada:** cantidad de topes, incumplimientos o costos.
- **Objetivo:** encontrar la configuración con el menor costo posible.

Problemas de modelado y simulación

Problema	Información conocida	Incógnita	Ejemplo
Modelado	Entradas y salidas observadas	Un modelo que explique los datos y generalice	Clasificador de imágenes
Simulación	Una entrada y un modelo	La salida producida por el modelo	Pronóstico meteorológico



La distinción importa porque la computación evolutiva puede buscar directamente una solución o puede ajustar los parámetros de un modelo.

Por qué la optimización aparece en toda la asignatura

Muchos problemas de modelado también se pueden reescribir como optimización:

$$\theta^* = \arg \min_{\theta} \mathcal{L}(\text{datos}, \text{modelo}_{\theta})$$

- En una red neuronal, θ son pesos y $\mathcal{L}$ puede ser el error de predicción.
- En un sistema difuso, θ puede representar parámetros de membresía o reglas.
- En esta unidad, θ será una solución candidata, como un horario, una ruta o una configuración.

La computación evolutiva es especialmente útil cuando la función se puede evaluar, pero no es fácil derivarla, es discontinua, ruidosa o proviene de una simulación.

Optimización combinatoria

Busca el mejor objeto dentro de un conjunto finito de objetos.

El conjunto puede ser finito y aun así ser imposible de recorrer exhaustivamente. Por ejemplo, ordenar n actividades tiene $n!$ posibilidades.

$$x^* = \arg \min_{x \in \Omega} f(x)$$

- Ω : soluciones admisibles o espacio de búsqueda.
- x : una configuración candidata.
- $f(x)$: función objetivo o medida de calidad.

La metaheurística administra un presupuesto limitado de evaluaciones de f .

Un problema de búsqueda tiene tres tareas

Tarea	Pregunta que se responde	Ejemplo: horario académico
Encontrar	¿Qué candidatos puedo construir?	Asignaciones de cursos, salas y bloques
Evaluar	¿Qué tan buena es una candidatura?	Topes, capacidad, preferencias, uso de salas
Aceptar	¿Es factible y debe conservarse?	Respetar restricciones y compite con otros

Antes de hablar de algoritmos, debemos hacer explícitos el espacio, la evaluación y las restricciones. Eso hace que el diseño sea discutible y reproducible.

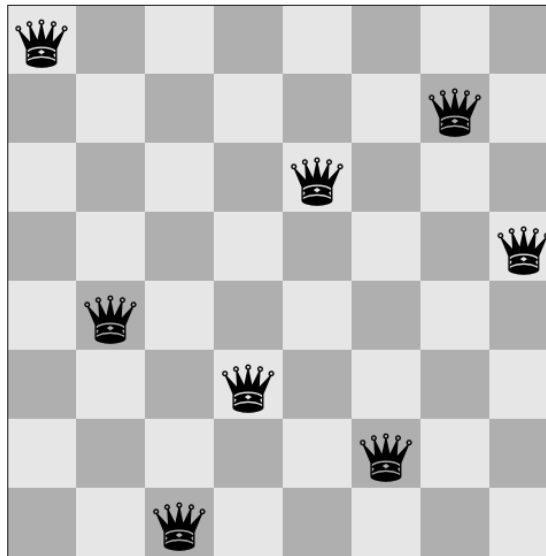
Las siglas distinguen objetivo y restricciones

	Objective function	
Constraints	Yes	No
Yes	Constrained optimisation problem	Constraint satisfaction problem
No	Free optimisation problem	No problem

Sigla	Nombre completo	Qué se busca
FOP	Problema de Optimización Libre, <i>Free Optimisation Problem</i>	Optimizar una función sin restricciones explícitas
CSP	Problema de Satisfacción de Restricciones, <i>Constraint Satisfaction Problem</i>	Encontrar una solución que cumpla todas las restricciones
CSOP / COP	Problema de Optimización y Satisfacción de Restricciones, <i>Constraint Satisfaction and Optimisation Problem</i> . También se llama Problema de Optimización Restringido, <i>Constrained Optimisation Problem</i>	Optimizar entre las soluciones que cumplen las restricciones

Ejemplo conductor: N reinas

Ubicar n reinas en un tablero $n \times n$ sin que dos se ataquen.



Dos reinas entran en conflicto si comparten fila, columna o diagonal. El ejemplo es pequeño, visual y permite separar con cuidado **objetivo**, **restricciones** y **representación** antes de implementar.

N reinas como FOP, CSP y CSOP

Formulación	Qué se define	Cuándo se resuelve
FOP	Una evaluación, como el número de conflictos	Al encontrar la configuración con el mejor valor
CSP	Predicados de factibilidad	Al encontrar una configuración que cumple todos los predicados
CSOP	Restricciones y una evaluación	Al obtener la mejor configuración entre las que son factibles

Por ejemplo, una formulación CSOP puede exigir una reina por fila y columna, y minimizar los conflictos diagonales:

$$\min D(s) \quad \text{sujeto a una reina por fila y columna}$$

Esta formulación también aparece en aplicaciones reales, donde algunas condiciones son obligatorias y otras expresan preferencias.

La representación ya es parte de la solución

Representar el tablero completo permite demasiadas configuraciones inútiles. Si imponemos una reina por columna, podemos usar un vector:

$$x = (x_1, \dots, x_n), \quad x_i = \text{fila de la reina en la columna } i$$

Una primera versión exige que no se repitan filas. Una versión aún mejor restringe directamente el espacio a **permutaciones** de $\{1, \dots, n\}$.

Para seis reinas, el vector $x = [5, 3, 1, 6, 4, 2]$ es una permutación. Esto garantiza una reina por fila y columna. Solo falta evaluar las diagonales.

Reducir el espacio sin perder soluciones relevantes

Con una permutación, el conflicto diagonal queda dado por:

$$C(x) = \sum_{i < j} \mathbf{1}\{|x_i - x_j| = |i - j|\}$$

- La representación elimina conflictos de fila y columna **por construcción**.
- $C(x)$ cuenta pares de reinas que comparten diagonal.
- $C(x) = 0$ indica una solución al CSP de N reinas.

La representación no es un detalle de programación: determina el tamaño del espacio, el costo de evaluar y qué operadores producen soluciones válidas.

Paradigmas de búsqueda

Aproximación sistemática

- Recorre el espacio siguiendo una estrategia exhaustiva.
- Puede garantizar óptimo o demostrar que no existe solución.
- Su costo suele crecer de forma prohibitiva.

Búsqueda local

- Examina una parte del espacio desde uno o pocos candidatos.
- Puede ser rápida y útil.
- No garantiza óptimo, factibilidad ni inexistencia de solución.

En IPD434 nos interesan métodos aproximados: hacen explícito el intercambio entre calidad, costo y tiempo de cómputo.

Construir o perturbar

Enfoque	Idea	Ejemplo de horario
Constructivo	Crear una solución desde un estado vacío	Asignar cursos uno a uno
Perturbativo	Modificar una solución existente	Cambiar una sala o bloque

La mayoría de las metaheurísticas usa ambos enfoques: necesita construir soluciones iniciales y luego producir variaciones para buscar mejoras.

Heurísticas: reglas que aprovechan estructura

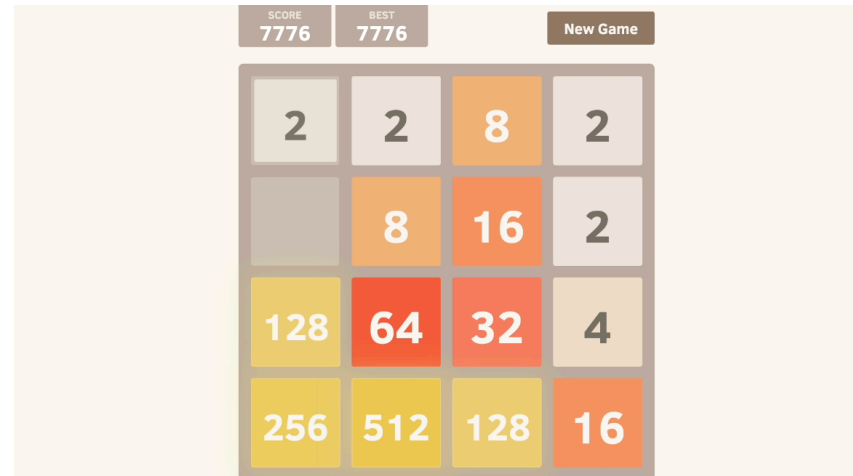
Una heurística es una regla práctica que orienta una decisión con información del problema.

Ejemplo de estrategia voraz: al construir un horario, asignar primero el curso con menor cantidad de bloques compatibles.

Ventaja	Límite
Puede obtener una solución razonable muy rápido.	Una decisión localmente buena puede bloquear una solución global mejor.

Una heurística puede generar una buena población inicial. También puede complementar el trabajo de una metaheurística.

Actividad breve: diseñar una heurística para 2048



Antes de proponer una regla, debemos precisar el objetivo. Puede ser obtener la ficha de mayor valor, mantener casillas libres o prolongar la partida.

Una heurística posible es conservar las fichas de mayor valor en una esquina y evitar movimientos que cierren el tablero.

La regla usa conocimiento del juego y permite decidir con rapidez. Sin embargo, una buena decisión inmediata no garantiza el mejor resultado al final de la partida.

Metaheurísticas: estrategias de búsqueda reutilizables

Proceso automatizado para encontrar una buena solución en un espacio grande, construyéndola o mejorándola, sin garantía de óptimo global.

No codifican reglas específicas de un único dominio. En cambio, establecen cómo generar, evaluar, aceptar y diversificar candidatos.

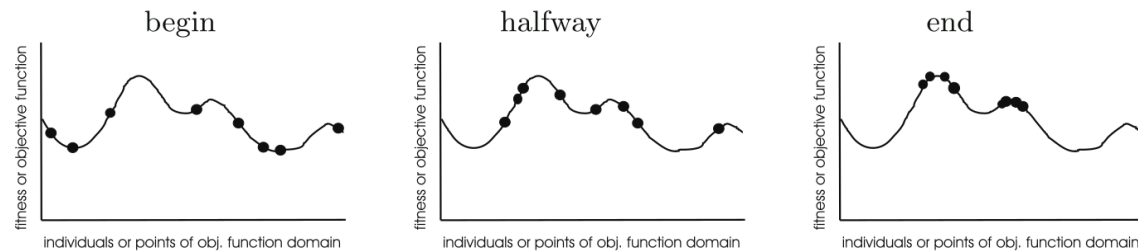
Heurística	Metaheurística
Decide con conocimiento particular del problema.	Define una estrategia de búsqueda aplicable a distintas formulaciones.
Ej.: priorizar asignaturas con mayor demanda.	Ej.: búsqueda local, recocido simulado o evolución.

Lenguaje mínimo de búsqueda local

- **Candidato:** solución que puede evaluarse.
- **Movimiento o perturbación:** cambio aplicado a un candidato.
- **Vecindario $N(x)$:** conjunto de candidatos alcanzables con un movimiento.
- **Criterio de selección:** decide qué vecino pasa a ser el siguiente candidato.
- **Reinicio:** introduce una nueva región cuando se detecta estancamiento.
- **Criterio de término:** limita evaluaciones, iteraciones, tiempo o ausencia de mejora.

Este vocabulario reaparecerá en los algoritmos evolutivos. La diferencia es que allí se trabaja con una **población** y no con un único candidato.

Exploración y explotación



- **Explotar (intensificar):** refinar regiones que ya parecen prometedoras.
- **Explorar (diversificar):** visitar regiones nuevas del espacio de búsqueda.

Explotar demasiado lleva a convergencia prematura en un óptimo local. Explorar demasiado impide consolidar mejoras. El equilibrio depende del problema, la representación y el presupuesto.

Metaheurísticas clásicas

Método	Mecanismo para evitar o manejar óptimos locales
Ascenso de colina	Acepta mejoras en el vecindario. Puede detenerse en un óptimo local.
Búsqueda local iterada	Perturba la solución y vuelve a ejecutar una búsqueda local.
Búsqueda tabú	Usa memoria para evitar movimientos o soluciones visitadas recientemente.
Recocido simulado	Acepta a veces una solución peor para escapar de un óptimo local.
GRASP	Construye una solución voraz con decisiones aleatorias y luego la mejora localmente.

No buscamos memorizar algoritmos aislados: buscamos reconocer cómo cada uno administra exploración, explotación y presupuesto.

Optimización con varios objetivos

En planificación académica, minimizar topes puede entrar en conflicto con minimizar salas o maximizar preferencias. Una suma ponderada los combina:

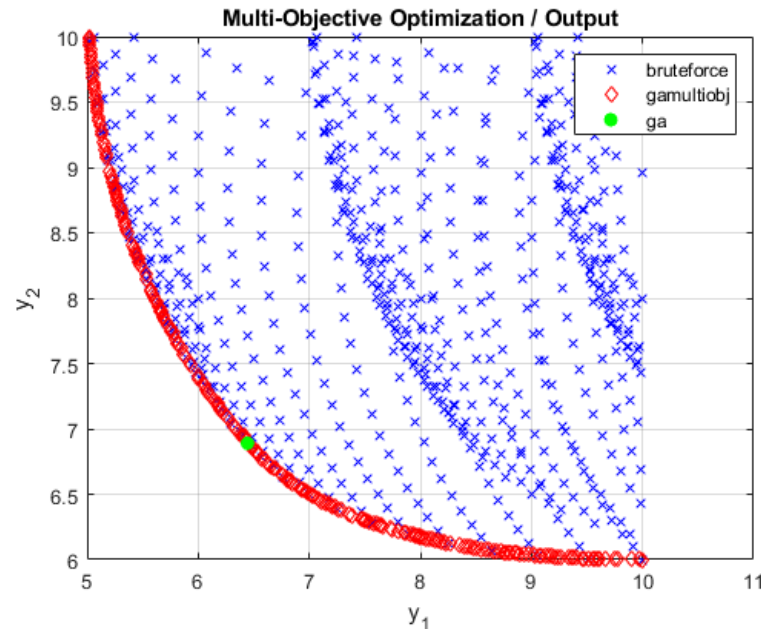
$$f(x) = w_1 f_1(x) + w_2 f_2(x) + \cdots + w_m f_m(x)$$

Los pesos incorporan una preferencia que debe justificarse. La optimización con varios objetivos mantiene el conflicto explícito y busca un conjunto de soluciones de compromiso.

Dominancia y frente de Pareto

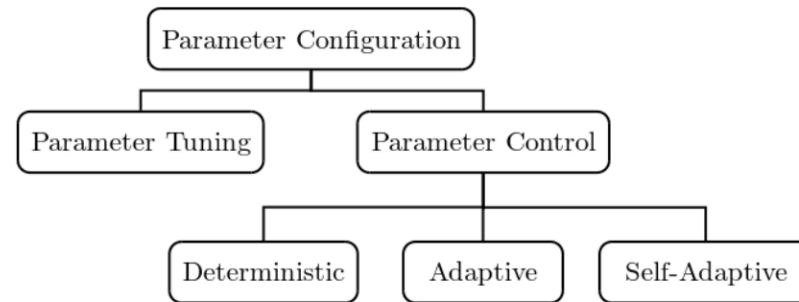
Para minimización, x domina a y si:

$$f_i(x) \leq f_i(y) \quad \forall i \quad \wedge \quad \exists j : f_j(x) < f_j(y)$$



Las soluciones no dominadas forman una aproximación al frente de Pareto. El algoritmo puede ofrecer alternativas. La elección final depende de las preferencias del contexto de aplicación.

Parámetros: definirlos antes o durante la búsqueda



- **Ajuste previo:** los valores se fijan antes de ejecutar, mediante pruebas o conocimiento previo.
- **Control durante la ejecución:** los valores cambian mientras el algoritmo trabaja.
 - *Determinista:* sigue una regla programada.
 - *Adaptativo:* responde al desempeño observado.
 - *Autoadaptativo:* los parámetros evolucionan junto con las soluciones.

Esta distinción será importante al discutir mutación, tamaño de población y presión de selección.

La transición clave: ¿por qué una población?

Una búsqueda local ve principalmente el vecindario de un candidato. Una población permite mantener varias hipótesis de solución al mismo tiempo.

Enfoque	Evolución de la búsqueda
Búsqueda local	Un candidato → una trayectoria
Búsqueda poblacional	Una población → varias trayectorias

La computación evolutiva usa diversidad poblacional como memoria de regiones distintas y la selección como mecanismo para concentrar recursos en las más prometedoras.

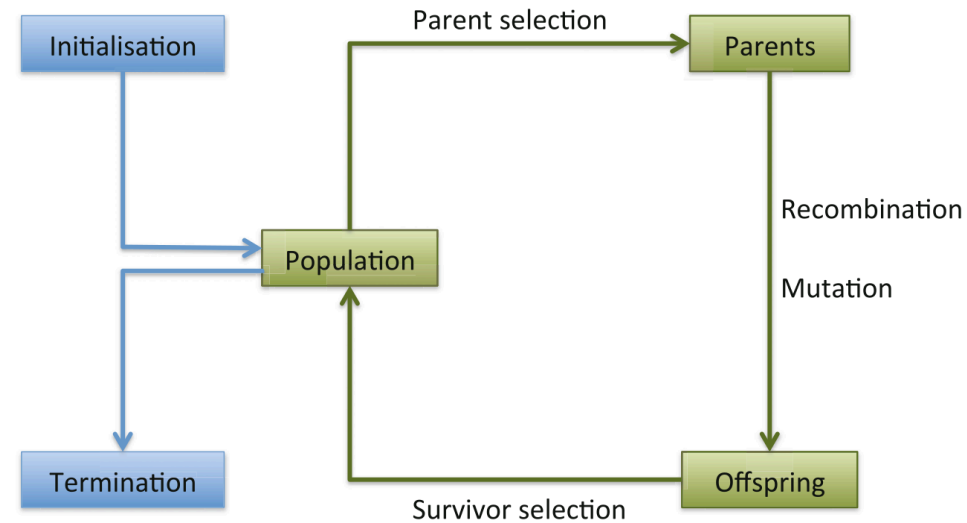
Idea central

Las técnicas evolutivas son metaheurísticas inspiradas en la evolución natural. Transforman una población de soluciones mediante selección, variación y supervivencia.

La evolución natural aporta la idea general. Sin embargo, la evaluación sigue siendo una decisión de ingeniería. La **aptitud** define qué significa que una solución esté bien adaptada al problema.

Biología	Optimización
Individuo	Solución candidata
Ambiente	Función de evaluación y restricciones
Reproducción	Generación de descendencia
Adaptación	Mejor desempeño según el objetivo

Ciclo general de un algoritmo evolutivo



$$P_t \longrightarrow P'_t \longrightarrow O_t \longrightarrow P_{t+1}$$

Selección de padres → variación → evaluación y supervivencia

Cada flecha representa una decisión. Cambiar la representación, los operadores o el reemplazo cambia el comportamiento del algoritmo.

Pseudocódigo: el orden importa

```
P ← inicializar población
evaluar P
mientras no se cumpla el criterio de término:
    padres ← seleccionar(P)
    hijos ← variar(padres)           # recombinación y/o mutación
    evaluar hijos
    P ← seleccionar_supervivientes(P, hijos)
retornar mejor solución observada
```

La calidad que vemos al final depende de **toda** esta cadena. Por eso, “usar un algoritmo genético” no es una especificación suficiente para reproducir un resultado.

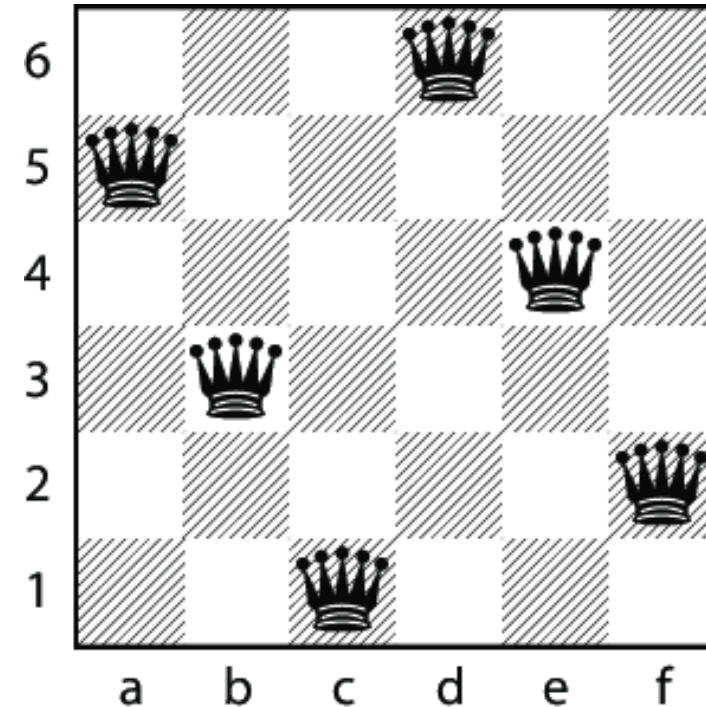
Genotipo, fenotipo y representación

Fenotipo

La solución interpretada en el dominio:
tablero, horario, ruta o asignación de recursos.

Genotipo

La codificación manipulada por el algoritmo:
bits, vector real, permutación o árbol.



Codificar lleva del fenotipo al genotipo.
Decodificar permite interpretar un genotipo
en el problema real.

Componentes de un individuo

Para el vector de seis reinas $X = [5, 3, 1, 6, 4, 2]$:

- **Individuo o cromosoma:** el vector completo X .
- **Gen:** una posición, por ejemplo $X[0]$, asociada a la primera columna.
- **Alelo:** su valor particular, aquí 5, que indica la fila de esa reina.

Estos nombres ayudan a describir los operadores. Cada cambio debe tener una interpretación útil y debe preservar o reparar las restricciones necesarias.

Una representación útil conecta con sus operadores

Una codificación debería:

- cubrir todas las soluciones relevantes.
- evitar, reparar o penalizar candidatos inválidos.
- admitir variaciones pequeñas con significado.
- permitir una evaluación eficiente.
- evitar redundancia innecesaria.

Un cruzamiento de un punto sobre dos permutaciones puede repetir filas y eliminar otras. Para N reinas se requieren operadores de permutación o una estrategia explícita de reparación.

Aptitud: medir la calidad de cada individuo

La función de **aptitud** o *fitness* permite comparar individuos. Para N reinas podemos minimizar directamente los conflictos diagonales:

$$f(x) = C(x)$$

Si una implementación requiere maximizar, una transformación posible es:

$$F(x) = \frac{1}{1 + C(x)}$$

La transformación debe preservar el orden relevante y no exagerar diferencias numéricas. En DEAP, el signo de los pesos permite declarar minimización o maximización sin cambiar artificialmente el objetivo.

Población y diversidad

Una población es un conjunto de candidatos en el que puede haber elementos repetidos. Su tamaño suele fijarse, pero una población grande no garantiza diversidad por sí sola.

Dónde medir diversidad	Pregunta
Genotipo	¿Los cromosomas son distintos?
Fenotipo	¿Representan soluciones distintas?
Aptitud	¿Obtienen calidades distintas?

Varios cromosomas diferentes pueden representar el mismo fenotipo. Por eso, la medida de diversidad debe corresponder al problema.

Diversidad: lo que conviene monitorear

Para una población binaria, la entropía en la posición j puede expresarse como:

$$H_j = -p_j \log p_j - (1 - p_j) \log(1 - p_j)$$

También se pueden usar distancia de Hamming, distancia euclidiana, individuos únicos o diversidad de objetivos.

Registrar solo la mejor aptitud no permite saber si la población convergió de forma adecuada o perdió diversidad demasiado pronto.

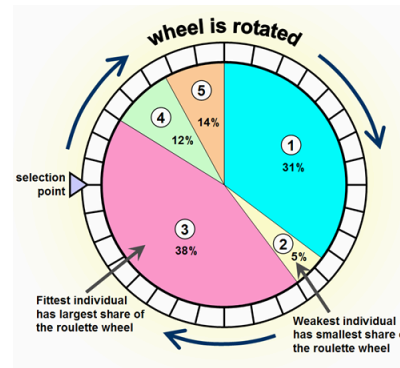
Operadores: tres decisiones diferentes

Tipo	Pregunta	Ejemplos
Selección de padres	¿Quién puede producir descendencia?	ruleta, ordenamiento, torneo
Variación	¿Cómo se producen nuevos candidatos?	recombinación, mutación
Supervivencia	¿Quién queda disponible en la próxima generación?	elitismo, ordenamiento, edad

Separar estas etapas evita una confusión frecuente: ser elegido como padre no implica sobrevivir, y sobrevivir no implica tener más hijos.

Selección de padres

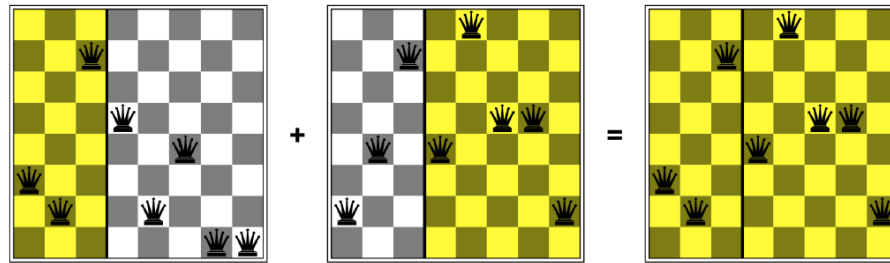
Método	Idea	Riesgo o control
Aleatoria	No usa la aptitud	Sirve como referencia, pero no aprovecha la calidad
Ruleta	Probabilidad proporcional a la aptitud	Es sensible a la escala y a valores extremos
Ordenamiento	Probabilidad según la posición ordenada	Controla la presión, pero pierde la magnitud absoluta
Torneo	Muestrea k y escoge el mejor	k regula la presión



Un torneo más grande aumenta la presión de selección y puede reducir la diversidad.

Recombinación: combinar información útil

La recombinación toma dos o más padres para formar descendencia. Es útil solo si los fragmentos heredados conservan significado en la representación.

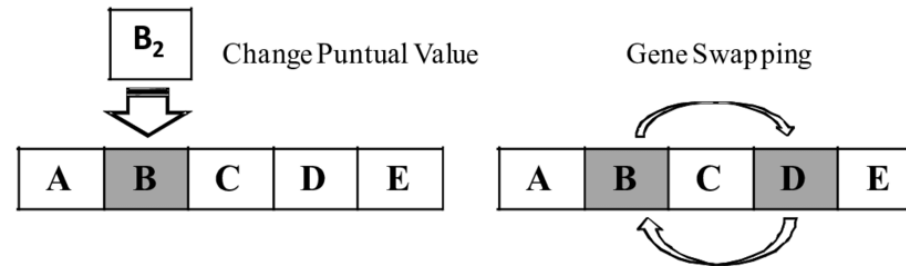


- Cadenas binarias: uno o varios puntos de corte.
- Vectores reales: combinaciones aritméticas o intermedias.
- Permutaciones: PMX, OX o CX, que evitan duplicados.

En N reinas, OX conserva un segmento de un padre y completa el resto según el orden del otro. El resultado sigue siendo una permutación válida.

Mutación: recuperar y abrir alternativas

La mutación es una perturbación estocástica aplicada a un individuo.



Representación	Mutaciones habituales
Bits	inversión de un bit
Vectores reales	ruido gaussiano
Permutaciones	intercambio, inserción, inversión

Una tasa muy alta se aproxima al muestreo aleatorio. Una tasa muy baja puede congelar la población. También se debe distinguir entre la probabilidad de mutar un individuo y la probabilidad de modificar cada gen.

Recombinación y mutación cumplen roles complementarios

Padres → recombinación → mutación → variantes evaluables

- La recombinación reutiliza y combina información ya presente en la población.
- La mutación introduce alternativas que la recombinación por sí sola podría no alcanzar.
- Ninguna garantiza una mejora. La aptitud se conoce después de evaluar.

En el lenguaje de las metaheurísticas, ambos son mecanismos de perturbación. La selección y el reemplazo determinan cuánto se aprovechan las soluciones que funcionaron bien.

Selección de supervivientes

Rank	$p_{survival}$	Rank	$p_{survival}$	Rank	$p_{survival}$	Rank	$p_{survival}$
1	0.100	6	0.075	11	0.045	16	0.020
2	0.095	7	0.070	12	0.040	17	0.015
3	0.090	8	0.065	13	0.035	18	0.010
4	0.085	9	0.060	14	0.030	19	0.005
5	0.080	10	0.055	15	0.025	20	0.000

- **Reemplazo generacional:** la descendencia sustituye toda la población.
- **Reemplazo estacionario:** se cambian pocos individuos por iteración.
- **Ordenamiento por aptitud:** padres e hijos compiten y permanecen los mejores.
- **Edad:** se limita cuántas generaciones puede permanecer un individuo.
- **Elitismo:** se preserva explícitamente uno o más mejores individuos.

Elitismo: ventajas y riesgos

Con elitismo, el mejor valor observado no empeora entre generaciones, porque al menos un buen individuo se conserva.

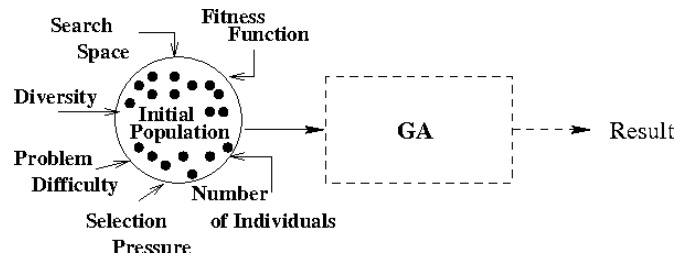
Sin embargo, demasiado elitismo produce copias y reduce diversidad:

Conservar el mejor valor ↔ mantener diversidad para adaptarse

La pregunta adecuada no es “¿usar elitismo?”, sino cuántos individuos preservar y qué mecanismo mantendrá suficiente exploración.

Cómo empezar y cómo terminar

Inicialización



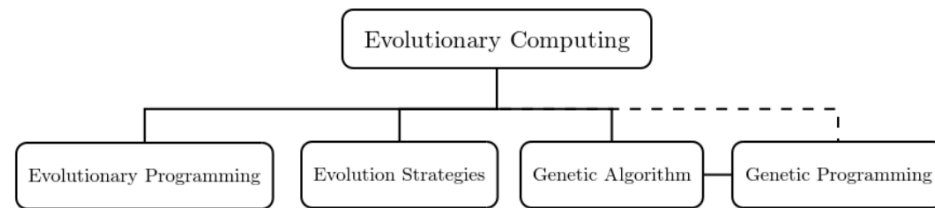
- aleatoria, para cubrir distintas regiones.
- heurística, para incluir buenas soluciones conocidas.
- híbrida, para equilibrar ambas.

Término

- presupuesto de evaluaciones.
- número de generaciones.
- tiempo máximo.
- calidad objetivo.
- estancamiento o pérdida de diversidad.

Para comparar configuraciones conviene usar el mismo presupuesto de **evaluaciones de aptitud**.

Cuatro familias históricas



Las fronteras modernas son flexibles. Aun así, las familias ayudan a reconocer combinaciones frecuentes de representación y operadores.

Comparación de familias evolutivas

Familia	Representación típica	Variación característica	Énfasis
Programación evolutiva (EP)	Vectores reales	Mutación gaussiana	Predicción y competencia, con poca recombinación en su forma histórica
Estrategias evolutivas (ES)	Vectores reales y parámetros	Recombinación y mutación	Autoadaptación y esquemas (μ, λ) / $(\mu + \lambda)$
Algoritmos genéticos (GA)	Cadenas, luego representaciones generales	Cruzamiento y mutación	Generaciones, selección y recombinación
Programación genética (GP)	Árboles o programas	Intercambio y cambio de subárboles	Evolución de expresiones o programas

En un informe técnico se deben indicar la representación y los operadores concretos. El nombre de la familia no entrega información suficiente para reproducir el método.

Estrategias de evolución: dos esquemas de supervivencia

Con μ padres y λ descendientes:

Esquema	Supervivientes
(μ, λ)	Solo compiten los λ hijos. Los padres desaparecen.
$(\mu + \lambda)$	Padres e hijos compiten juntos. Este esquema permite elitismo.

Esta distinción conecta directamente con la selección de supervivientes vista antes: el diseño del reemplazo modifica tanto presión de selección como diversidad.

Derivaciones y métodos inspirados en la naturaleza

Coevolución

Una o varias poblaciones se evalúan en interacción. Puede modelar cooperación (simbiosis) o competencia (parasitismo).

La calidad de una solución puede depender de con quién se compara, no solo de una función fija.

Otras metaheurísticas poblacionales

- **Ant Colony Optimization:** la información colectiva se expresa mediante feromonas.
- **Particle Swarm Optimization:** los candidatos se mueven guiados por experiencia propia y del grupo.

Comparten la idea poblacional, pero sus mecanismos no son los de un algoritmo genético.

Actividad: algoritmo evolutivo para N reinas

Diseñe un EA que encuentre una permutación con $C(x) = 0$.

Decisión	Propuesta inicial razonable
Individuo	Permutación de $\{1, \dots, n\}$
Aptitud	Número de conflictos diagonales $C(x)$, a minimizar
Padres	Torneo
Recombinación	OX u otro operador de permutaciones
Mutación	Intercambio de dos posiciones
Supervivencia	Generacional con elitismo moderado
Término	Éxito, presupuesto de evaluaciones o estancamiento

No son elecciones universales: son una hipótesis de diseño que debemos poner a prueba.

Implementación: la biblioteca no reemplaza el diseño

El material práctico complementario, [03_DEAP.ipynb](#) y [notebook/03_ga_n_reinas.ipynb](#), permite implementar el ciclo usando DEAP.

DEAP separa:

- definición de individuos y aptitud.
- registro de operadores.
- ejecución del ciclo evolutivo.
- estadísticas y registro experimental.

La biblioteca puede ejecutar los operadores, pero no decide una representación válida, qué objetivos importan ni si una comparación experimental es justa.

Protocolo experimental mínimo

1. Fijar instancia, restricciones y presupuesto de evaluaciones.
2. Definir una línea base: muestreo aleatorio o heurística simple.
3. Ejecutar varias semillas independientes.
4. Registrar la mejor aptitud, el promedio, la dispersión y la diversidad por generación.
5. Reportar éxito, costo computacional y distribución de resultados.
6. Comparar curvas respecto de evaluaciones, no solo respecto de generaciones.

La tasa de éxito es la proporción de ejecuciones que alcanzan $C(x) = 0$.

Qué significa una comparación justa

Práctica insuficiente	Práctica recomendable
Informar la mejor corrida.	Informar mediana, dispersión y tasa de éxito.
Comparar generaciones con poblaciones de tamaños distintos.	Igualar evaluaciones de aptitud o declarar claramente el presupuesto.
Cambiar todos los parámetros a la vez.	Realizar análisis de sensibilidad controlado.
Usar una semilla implícita.	Registrar semillas, instancia y configuración.

Un algoritmo evolutivo es estocástico. Una ejecución aislada solo muestra un resultado posible. Varias ejecuciones permiten estudiar el comportamiento habitual y su variabilidad.

Preguntas para discutir en clase

1. ¿Qué se pierde y qué se gana al restringir N reinas a permutaciones?
2. ¿Qué pasaría si usamos cruzamiento de un punto sin reparación?
3. ¿Cómo se comparan población 50 por 100 generaciones y población 100 por 50 generaciones?
4. ¿Qué señal mostraría que el elitismo está causando convergencia prematura?
5. En un horario, ¿qué restricciones son duras y qué preferencias deberían ser objetivos?

Estas preguntas conectan la implementación con decisiones de modelado, que es el aprendizaje central de la unidad.

Síntesis

formular → representar → evaluar → variar y seleccionar → medir con evidencia

- Un algoritmo evolutivo administra un presupuesto de evaluaciones sobre una población.
- Representación, aptitud, operadores y supervivencia forman un diseño inseparable.
- La selección impulsa la explotación. La variación y la diversidad sostienen la exploración.
- Los resultados deben evaluarse con repeticiones, líneas base y presupuestos comparables.

La próxima unidad abordará otra manera de optimizar: ajustar parámetros de modelos a partir de datos mediante aprendizaje y gradiente.

Referencias y atribución del material

Material base de la asignatura

- **Gálvez Ramírez, Nicolás.** *Introducción a la Computación Evolutiva*, material docente de IPD434, Universidad Técnica Federico Santa María. Archivo base: [03_ComputacionEvolutiva.ipynb](#) .
- Este documento adapta, reorganiza y amplía ese material para fortalecer las conexiones conceptuales y el contexto de la asignatura.
- Material práctico complementario: [03_DEAP.ipynb](#) y [notebook/03_ga_n_reinas.ipynb](#) .

Referencias bibliográficas

- Eiben, A. E. y Smith, J. E. *Introduction to Evolutionary Computing*, 2.^a ed., Springer, 2015.
- Goldberg, D. E. *Genetic Algorithms in Search, Optimization, and Machine Learning*, Addison-Wesley, 1989.
- Deb, K. *Multi-Objective Optimization Using Evolutionary Algorithms*, Wiley, 2001.
- Fortin, F.-A. et al. "DEAP: Evolutionary Algorithms Made Easy", *Journal of Machine Learning Research*, 2012.